AWS | STUDENT PROJECT REPORT

# VPC Peering

Connect two VPCs. Prove private connectivity.

| | |
|---|---|
| Student name | **Anmoldeep Singh** |
| Batch | AWS Internship Batch 1 |
| Project title | AWS VPC Peering (Project 01) |
| Region | Mumbai / ap-south-1 |
| Completion date | 15 / 10 / 2026 |
| Email | singhanmoldeep2005@gmail.com |

| | | |
|---|---|---|
| VPC A (ewb-vpc-a) | 10.10.0.0/16 | Public 10.10.1.0/24 (bastion) \| Private 10.10.2.0/24 (App-A) |
| VPC B (ewb-vpc-b) | 10.20.0.0/16 | Private 10.20.1.0/24 (App-B) \| no internet gateway |
| Peering | ewb-a-b-peer | pcx-requester (A requester, B accepter) |

01 / ARCHITECTURE AND INVENTORY

# Architecture, CIDRs and resource inventory

One internet entry point (the bastion in VPC A). Two private application servers, one per VPC, connected only through the peering connection. Administration path: laptop > bastion > App-A or App-B (SSH ProxyJump). Project traffic: App-A <> App-B over private IPs.

```
  Laptop --SSH/22--> [Internet Gateway ewb-igw-a]
                          |
+---------- VPC A 10.10.0.0/16 ----------+      +------ VPC B 10.20.0.0/16 ------+
| Public  10.10.1.0/24  ewb-bastion .10  |      |  (no internet gateway)          |
|        |  ProxyJump ------------------------------> App-B                       |
| Private 10.10.2.0/24  ewb-app-a   .10  <==== pcx ====>  10.20.1.0/24 ewb-app-b .10 |
+----------------------------------------+      +---------------------------------+
```

## Why these CIDRs?

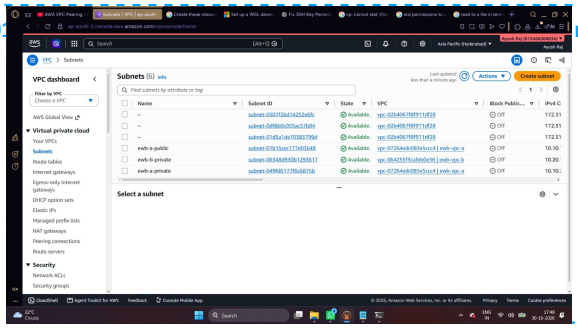
10.10.0.0/16 and 10.20.0.0/16 do not overlap, which is mandatory for peering. Each /24 subnet sits inside its own VPC range. Different subnet ranges alone would not fix overlapping VPC CIDRs.

## Resource inventory (fill actual IDs)

| Resource / Name tag | Location | CIDR / private IP | Actual ID |
|---|---|---|---|
| ewb-vpc-a | VPC A | 10.10.0.0/16 | vpc- |
| ewb-a-public | VPC A | 10.10.1.0/24 | subnet- |
| ewb-a-private | VPC A | 10.10.2.0/24 | subnet- |
| ewb-vpc-b | VPC B | 10.20.0.0/16 | vpc- |
| ewb-b-private | VPC B | 10.20.1.0/24 | subnet- |
| ewb-igw-a | attached to VPC A | - | igw- |
| ewb-rt-a-public / -a-private / -b-private | route tables | - | rtb- / rtb- / rtb- |
| ewb-sg-bastion / -app-a / -app-b | security groups | - | sg- / sg- / sg- |
| ewb-bastion | ewb-a-public | 10.10.1.10 + public IPv4 _____ | i- |
| ewb-app-a | ewb-a-private | 10.10.2.10, no public IP | i- |
| ewb-app-b | ewb-b-private | 10.20.1.10, no public IP | i- |
| ewb-a-b-peer | A to B | - | pcx- |

All resources tagged Project = EWB-VPC-Peering and Student = Anmoldeep Singh. Key pair: ewb-lab (private key kept only on my laptop).

EVIDENCE: Checkpoint 01 — route table shows only local route



02 / SUBNETS AND ROUTE TABLES

# Subnet CIDRs and the three route tables

### ewb-rt-a-public / associated to ewb-a-public

| Destination | Target | Purpose |
|---|---|---|
| 10.10.0.0/16 | local | Bastion to App-A |
| 0.0.0.0/0 | igw-ID (ewb-igw-a) | Bastion internet access |
| 10.20.0.0/16 | pcx-ID | Bastion SSH to App-B (ProxyJump) |

### ewb-rt-a-private / associated to ewb-a-private

| Destination | Target | Purpose |
|---|---|---|
| 10.10.0.0/16 | local | Existing VPC A route |
| 10.20.0.0/16 | pcx-ID | App-A to VPC B |

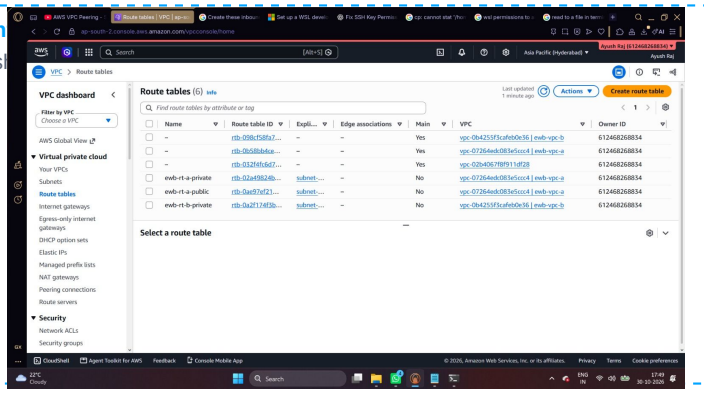
### ewb-rt-b-private / associated to ewb-b-private

| Destination | Target | Purpose |
|---|---|---|
| 10.20.0.0/16 | local | Existing VPC B route |
| 10.10.0.0/16 | pcx-ID | Return path to VPC A |

Note: the private route table has no 0.0.0.0/0 route and VPC B has no internet gateway, so neither application server has internet egress. Peering is not transitive, so VPC B cannot use VPC A's internet gateway.

**EVIDENCE: Checkpoin**

Paste your own screensh



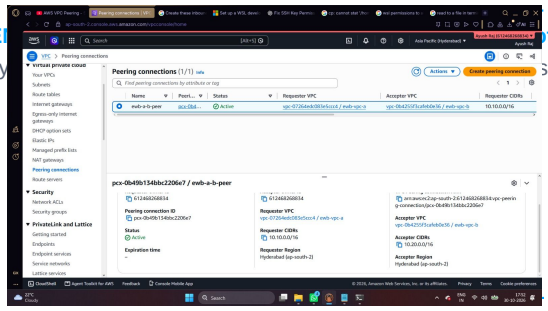
03 / PEERING AND SECURITY

# Active peering and inbound security group rules

| Field | Value |
|---|---|
| Name | ewb-a-b-peer |
| Requester VPC | ewb-vpc-a (10.10.0.0/16) |
| Accepter VPC | ewb-vpc-b (10.20.0.0/16) |
| Account / Region | My account / This Region (ap-south-1) |
| Status | Active (requested, then accepted) |

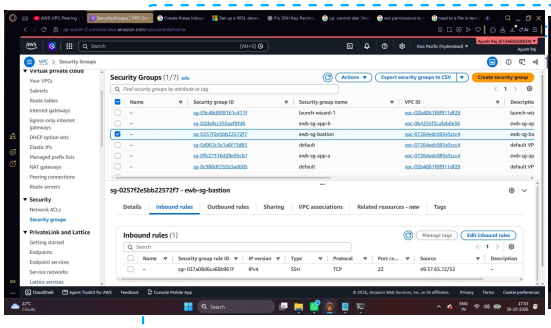
EVIDEN th VPC IDs

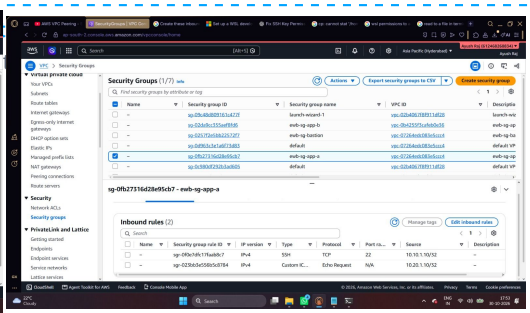
Paste y st and result).

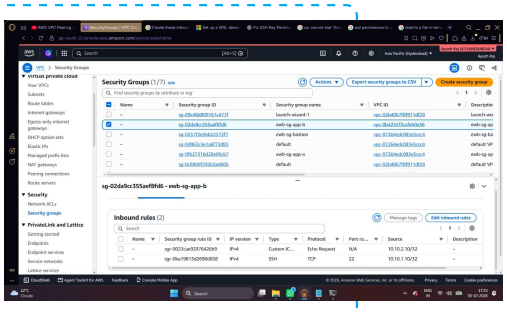


## Inbound rules (outbound left at default allow-all; NACLs default allow-all)

| Security group | Type / port | Source | Why |
|---|---|---|---|
| ewb-sg-bastion | SSH / TCP 22 | My public IPv4 /32 | Only I can reach the single public entry point |
| ewb-sg-app-a | SSH / TCP 22 | 10.10.1.10/32 | Admin only via the bastion |
| ewb-sg-app-a | ICMP Echo request (type 8) | 10.20.1.10/32 | Allow ping from App-B only |
| ewb-sg-app-b | SSH / TCP 22 | 10.10.1.10/32 | App-B sees the bastion's private IP as source |
| ewb-sg-app-b | ICMP Echo request (type 8) | 10.10.2.10/32 | Allow ping from App-A only |







04 / VERIFICATION

# Baseline, forward/reverse ping and private SSH

Local ~/.ssh/config used (all commands run on my laptop; key never copied to an instance):

```
Host ewb-bastion
    HostName <BASTION_PUBLIC_IP>
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

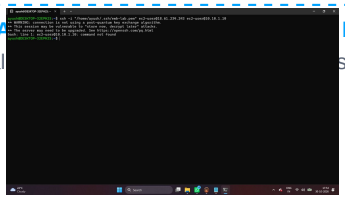
Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

| Test | Where / command | Expected (acceptance criteria) |
|---|---|---|
| 01 Baseline | App-A, before peering: ping -c 4 10.20.1.10 | No replies |
| 02 Forward | App-A, after routes: ping -c 4 10.20.1.10 | 4 replies; 0% loss |
| 03 Reverse | App-B, after routes: ping -c 4 10.10.2.10 | 4 replies; 0% loss |
| 04 Private SSH | Laptop: ssh ewb-app-b | Shell on App-B via bastion |
| 05 Isolation | EC2 console: App-A and App-B details | No public IPv4 on either |

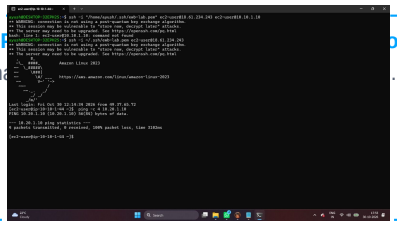
**EVIDENCE: Test 01 - BASELINE FA** **before peering (hostname, hostname -I, ping summary)**

Paste your own screenshot / terminal sult).



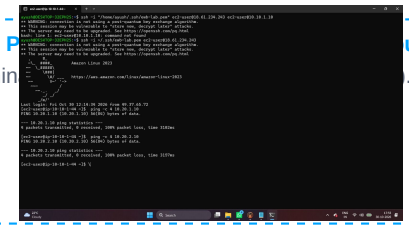
**EVIDENCE: Test 02 - FORWARD P** **utes**

Paste your own screenshot / termina .



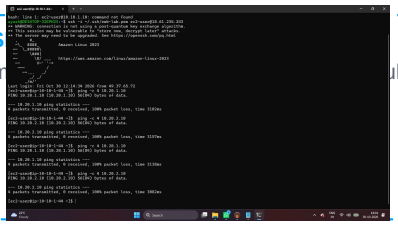
**EVIDENCE: Test 03 - REVERSE P** **utes**

Paste your own screenshot / termin .



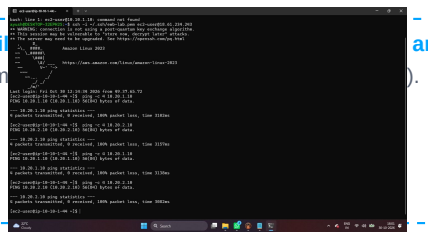
**EVIDENCE: Test 04 - App-B SS**

Paste your own screenshot / term ult).



**EVIDENCE: Test 05 - EC2 detail** **and App-B**

Paste your own screenshot / term ).



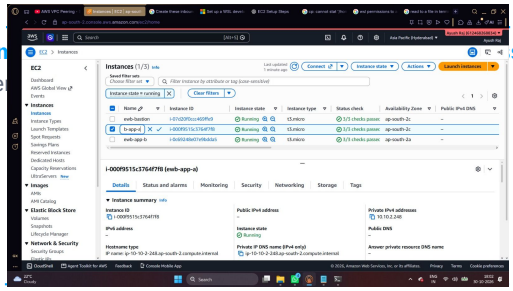
05 / CONTROLLED FAILURE

# ICMP failure and recovery, with explanation

Procedure: remove only the App-B inbound ICMP Echo request rule (source 10.10.2.10/32), ping from App-A again (replies stop), restore the exact same rule, ping again (replies resume). The SSH rule stays in place throughout.

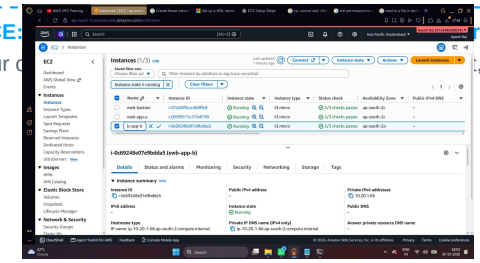
**EVIDENCE: Failure: ping fron** **S)**

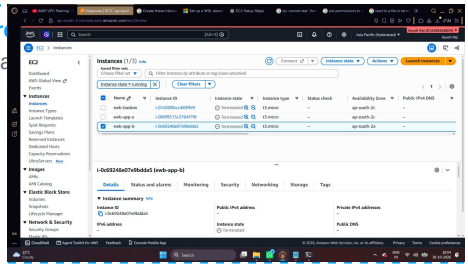
Paste your own screenshot / te



**EVIDENCE:** **restor**

Paste your , test a





## Short explanation

The peering connection and both routes stayed Active/unchanged during the test, so the network path existed the whole time. Only the destination security group changed. Removing the Echo-request rule made App-B drop the packets, which proves that routes decide *where* traffic can go while security groups decide *whether* it is allowed. Restoring the exact rule brought replies back (security groups are stateful, so replies need no extra rule).

## Forward path and return path

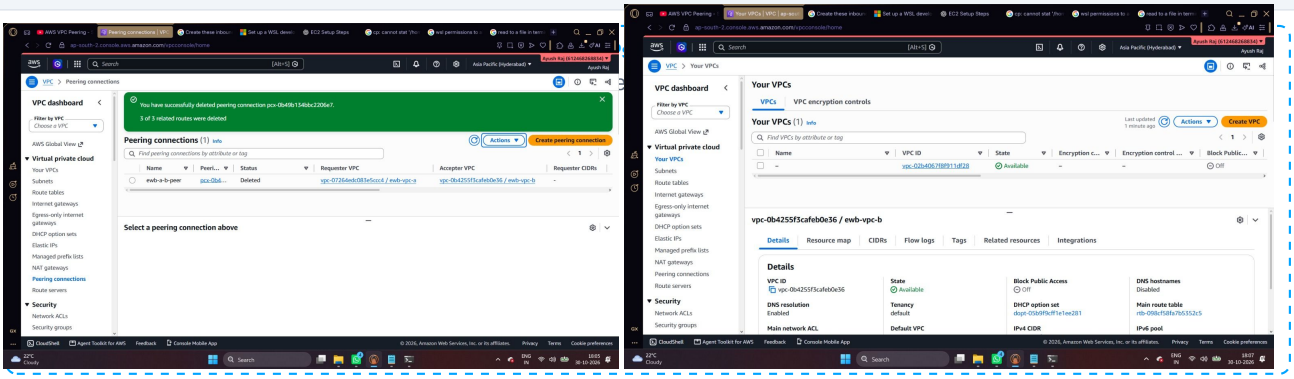
**Forward (App-A to App-B):** App-A (10.10.2.10) looks up 10.20.1.10 in ewb-rt-a-private, matches 10.20.0.0/16 and sends it to the peering connection; the packet reaches App-B, where ewb-sg-app-b allows Echo request from 10.10.2.10/32.

**Return (App-B to App-A):** App-B looks up 10.10.2.10 in ewb-rt-b-private, matches 10.10.0.0/16 to the same peering connection, and the stateful security group lets the reply out. Without the VPC B route the reply has no path back and ping fails.

07 / CLEANUP

# Cleanup evidence and what I learned

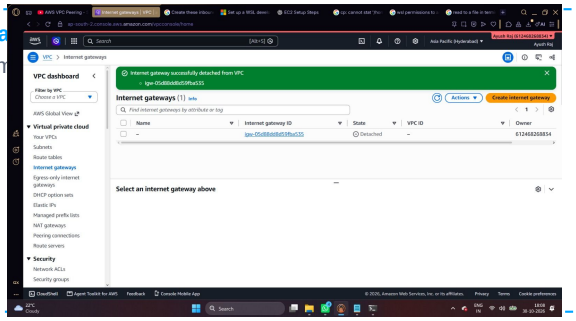
| Step | Done? |
|---|---|
| Terminated ewb-bastion, ewb-app-a, ewb-app-b; EBS volumes deleted | [ ] |
| No Elastic IPs left allocated | [ ] |
| Peer routes and ewb-a-b-peer deleted | [ ] |
| Security groups, subnets, route tables deleted | [ ] |
| ewb-igw-a detached and deleted; both VPCs deleted | [ ] |
| Key pair ewb-lab and local .pem removed | [ ] |
| Filtered by tag Project = EWB-VPC-Peering: nothing remains; Billing checked | [ ] |



## What I learned

**EVIDENCE: Your own short lea**

Paste your own screenshot / term



TyIn this project I learned that VPC peering is only the link between two VPCs.
For traffic to flow, the peering connection must be Active, both VPCs need a route to the other VPC's CIDR, and the destination security group must allow the traffic.
Before peering, the ping from App-A to App-B failed.
After I added the routes and accepted the connection, it worked in both directions.
When I removed the ICMP rule from the App-B security group, the ping stopped even though the routes were fine,
so security groups can block traffic on their own.
I also learned that the bastion needs a route to VPC B,
because with ProxyJump the bastion opens the SSH connection to App-B itself.
Finally, I saw that peering does not give VPC B internet access,
and that deleting the resources afterwards avoids charges.